התרגיל 6: Abstract data type

תאריך פרסום : 29.2.24

תאריך הגשה : 18.3.24 בשעה 23:59

מתרגלת אחראית : אסראא נסאסרה

משקל תרגיל : 4 נקודות

הנחיות לתרגיל :

- אין לייבא ספריות, חיצוניות או מובנות (למעט functools).
- שלד המחלקות בתרגיל בית זה יינתן לכם בנוסף למבני הנתונים הנדרשים. שימו לב כי ייתכן ויהיו הבדלים במימוש מבני הנתונים העומד לרשותכם לעבודת בית זו לעומת המימוש שראיתם בכיתה. **אין לערוך שינויים על מבני נתונים אלה**.

תיקיית המטלה תכיל בנוסף לקובץ המטלה עצמו (ה-pdf) את רשימת הקבצים הבאים :

קובץ ADTs.py אשר יכיל מימוש כל מבני הנתונים העומדים לרשותכם (Queue, Stack, ו-LinkedList).

3 הקבצים של שאלה 1 : Jop.py, CPUNodes.py, ו-JopRequestSystem.py.

קובץ BinaryTree.py של שאלה 2.

4 קבצי דוגמאות הרצה : example_Job.txt, example_CPUNodes.txt, example_JobRequestSystem.txt, example_BinaryTree.txt

- מומלץ לקרוא את כל העבודה **לפני** תחילת המימוש. אנו ממליצים להבין היטב את ארכיטקטורת המערכת ולתכנן אותה על דף נייר לפני תחילת כתיבת הקוד.
- **בתרגיל זה עליכם לבדוק תקינות קלט רק כאשר יש דרישה ברורה בשאלה**. בהיעדר דרישה זו הניחו כי הקלט תקין. במידה והקלט לא תקין עליכם לזרוק חריגה מתאימה בהתאם לדרישת השאלה\הסעיף. אינכם נדרשים לכתוב docstring בעבודה זו.
- על מימושכם לרוץ גם כאשר הקלטים אינם תקינים ולהציג הודעה מתאימה למשתמש כאשר הקלט אינו תקין. אין להשתמש במבני נתונים נוספים מעבר למבני הנתונים שנדרשתם. פתרונות אשר יעשו שימוש במבני נתונים אחרים (גם אם מדובר במבני נתונים מובנים בפייתון) ייפסלו.
- אין להוסיף שדות פרט לנדרש מכם בכל מחלקה אך אתם רשאים להוסיף שיטות עזר למחלקות השונות.

שאלה 1:

בשאלה זו תממשו מערכת לניהול משאבי מעבדים (CPUs) מרחוק המאפשרת לבקש שריון משאבים מוגדרים להרצת סקריפטים (לצורך השאלה - "בקשה") ע"י חלוקה המשאבים לפי זמינות וצורך. נגדיר כל בקשה כזו ע"י ארבעה פרמטרים: מספר ייחודי, תיאור הבקשה, דירוג הבקשה, וכמות המשאבים הדרושים לשריונה. מערכת ניהול זו (JobRequestSystem) מקבלת בקשה לשריון משאב (או משאבים). אם כמות המשאבים הזמינים העומדת לרשות המערכת מספיקה לשריון הבקשה החדשה אז המערכת מקצה אותם לטיפול בבקשה. אם משאבי המערכת אינם זמינים, הבקשה בעלת הדירוג המירבי תקבל את המשאבים ראשונה בעת זמינותם לאחר שחרורם מבקשות קודמות. עליכם לממש את המערכת לפי עקרונות תכנות מונחה-עצמים, תוך כדי שימוש במבני הנתונים המתאימים והיעילים ובהתאם לדרישות השאלה.

עליכם לממש את המחלקות הבאות: ***Job***, המייצגת בקשה לשריון משאב (או משאבים), את המחלקה ***CPUNodes***, המייצגת את המשאבים העומדים לרשות המערכת, ואת המחלקה ***JobRequestSystem***, המבצעת את ניהול המשאבים, שריונם לבקשות המתאימות וביצוע הבקשות בפועל.

סעיף א':

ממשו את המחלקה "בקשה"- ***Job*** המכילה את ארבעת השדות הבאים:

- שדה סטטי בשם last_job_id הוא שדה של המחלקה (ולא של המופע). השדה הינו בעל ערך התחלתי 0 אשר יתעדכן עם כל מופע שנייצר מהמחלקה Job.
- job_id הוא שדה פרטי מטיפוס int, מספר שאינו שלילי המייצג מזהה ייחודי לכל מופע מהמחלקה Job.
- description הוא שדה פרטי מטיפוס מחרוזת המייצג תיאור לבקשה. שדה זה יכול להיות מחרוזת ריקה.
- rank הוא שדה פרטי, מספר טבעי המייצג את דירוג הבקשה. ערך גבוה מייצג בקשה דחופה יותר.
- השדה job_requested_resources הוא שדה פרטי מטיפוס int, מספר טבעי המייצג את כמות המשאבים שיש לשריין עבור בקשה זו.

1) ממשו את בנאי המחלקה אשר מקבל את 3 הארגומנטים job_description, job_rank ו-job_request_resources ומאתחל את השדות לפי הארגומנט המתאים, וגם את השדה job_id. אם הקלט אינו מהטיפוס המתאים או שערכו אינו תקין עליכם לזרוק את החריגה TypeError או את החריגה ValueError, בהתאמה, בצירוף הודעה מתאימה.

```
def __init__(self, job_description, job_rank,
             job_request_resources):
```

2) עליכם לדרוס את האופרטור __repr__ כך שכעת הוא יחזיר תיאור לפרטי הבקשה לפי הפורמט הבא:

"ID: {job_id}, Resources: {job_requested_resources}, Rank: {rank}"

כאשר אדום מסמן את התווים הקבועים ושחור את ערכי השדות המתאימים.

3) ממשו את השיטה get_description המחזירה את השדה description.

```
def get_description(self):
```

4) ממשו את השיטה get_id המחזירה את השדה job_id של המופע ממחלקת Job.

```
def get_id(self):
```

5) ממשו את השיטה get_job_requested_resources המחזירה את השדה job_requested_resources, כמות המשאבים שיש לשריין עבור בקשת משאבים זו.

```
def get_job_requested_resources(self):
```

6) ממשו את השיטה get_rank אשר מחזירה את שדה הדירוג rank עבור בקשת שריון משאבים זו.

```
def get_rank(self):
```

7) ממשו את השיטה set_rank המקבלת את הארגומנט new_rank ומעדכנת את השדה rank. בדקו תקינות קלט באופן דומה לדרישות בשיטת הבנאי של המחלקה (סעיף א.1).

```
def set_rank(self, new_rank):
```

8) ממשו את השיטה set_description המעדכנת את השדה description ומשנה את ערכו לפי הארגומנט new_description. בדקו תקינות קלט באופן דומה לדרישות בשיטת הבנאי של המחלקה (סעיף א.1).

```
def set_description(self, new_description):
```

9) ממשו את השיטה set_job_requested_resources המעדכנת את כמות המשאבים שיש לשריין לבקשה זו בשדה job_requested_resource, בהתאם לארגומנט new_job_requested_resources. בדקו תקינות קלט באופן דומה לדרישות בשיטה הבנאי של המחלקה (סעיף א.1).

```
def set_job_requested_resources(self, new_job_requested_resources):
```

10) עליכם לממש את השיטות המאפשרות תמיכה בשש פעולות ההשוואה. ההשוואה בין מופעים שונים מטיפוס Job מוגדרת לפי שדה הדירוג שלהם rank.

דוגמאות הרצה למחלקה זו יצורפו בקובץ המיועד לכך – example_Job.txt

סעיף ב':

המשאבים העומדים לרשות מערכת ניהול המשאבים (***JobRequestSystem***) מיוצגים ע"י מופע מהמחלקה ***CPUNodes***. בכל מופע של ***CPUNodes*** יש מבנה נתונים מטיפוס רשימה מקושרת המהווה את המשאבים "בפועל". כל חוליה ברשימה המקושרת מייצגת "צרור של מספר משאבים".

ממשו את המחלקה ***CPUNodes*** המייצגת את המשאבים העומדים לרשות מערכת ניהול המשאבים. למחלקה ***CPUNodes*** יש את 3 **השדות הפרטיים** הבאים:

- השדה nodes_number מספר טבעי המייצג את מספר הצרורות של משאבים שיהיו למערכת.
- השדה capacity מספר טבעי המייצג את סה"כ כמות המשאבים בכל הצרורות של המערכת. ערכו של השדה capacity שווה לכל הפחות לשדה nodes_number. כלומר, אם ערכו של השדה nodes_number שווה ל-6 אז השדה capacity חייב להיות לכל הפחות 6.
- השדה cpu_nodes מטיפוס רשימה מקושרת (LinkedList). שדה זה מייצג את המשאבים העומדים לרשות המערכת "בפועל". אורכה של הרשימה המקושרת נקבע לפי השדה nodes_number. כל חוליה ברשימה המקושרת מייצגת "צרור של משאבים" ומכילה כערך רשימה מובנית של פייתון המורכבת משני איברים מטיפוס int, האיבר הראשון מייצג את כמות המשאבים שצרור זה מכיל והאיבר השני מייצג את כמות המשאבים הזמינים לשריון בצרור בכל רגע נתון.

1) ממשו את הבנאי של המחלקה אשר מקבל את שני הארגומנטים capacity ו- nodes_number ומאתחל את שני השדות בהתאמה. השדה הפרטי cpu_nodes מאותחל גם כן ע"י יצירת הרשימה המקושרת לפי הכללים הבאים:

i. אם השדה capacity מתחלק ללא שארית בשדה nodes_number:

i.i הוסיפו לרשימה המקושרת בשדה cpu_nodes חוליות כמספר השדה nodes_number. ערכה כל חוליה יאותחל ברשימה מובנית של שני איברים כאשר שני האיברים שווים ל:
[capacity / nodes_number, capacity / nodes_number]

ii. אחרת:

ii.i הוסיפו לרשימה המקושרת בשדה **cpu_nodes** חוליות כמספר השדה **nodes_number**. החוליות ה- **nodes_number-1** הראשונות יאותחלו כל אחת עם רשימה בעלת שני איברים כאשר שני האיברים שווים ל:

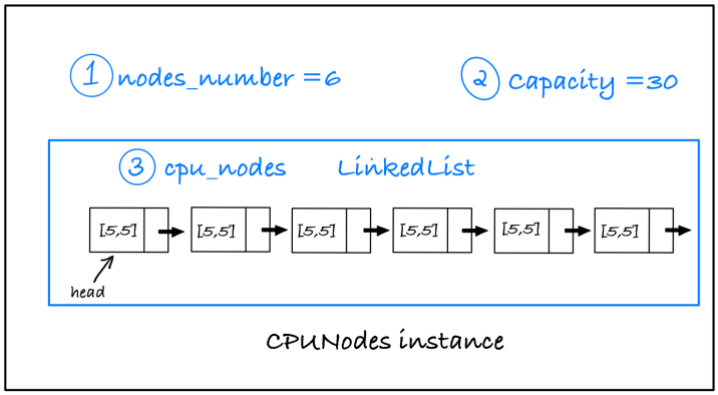
[capacity // nodes_number, capacity // nodes_number]

ii.ii את החוליה האחרונה יש לאתחל עם הערך:
[capacity//nodes_number+capacity%nodes_number,
capacity//nodes_number+capacity%nodes_number]

בדקו תקינות קלט בהתאם לדרישות המחלקה. אם שני הארגומנטים שהבנאי מקבל אינם מהטיפוס הנדרש, יש לזרוק את החריגה TypeError. אם ערכיהם של שני הארגומנטים לא עומדים בדרישות המחלקה CPUNodes יש לזרוק את החריגה ValueError. זרקו את החריגות בצירוף הודעה מתאימה לסוג החריגה.

```
def __init__(self, capacity, nodes_number):
```

דוגמה:
להלן איור סכמתי למופע מהמחלקה ***CPUNodes*** אשר אותחל עם capacity בערך 30 ו-nodes_number בערך 6:

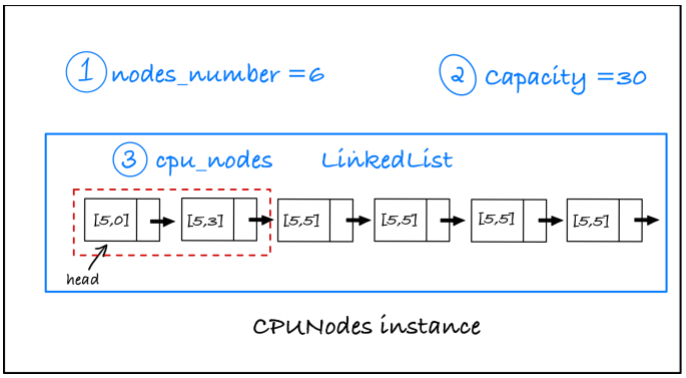


2) ממשו את השיטה get_cpu_capacity המחזירה את ערכו של השדה capacity. למשל בהינתן אותו מופע באיור הסכמתי, השיטה תחזיר 30.

```
def get_cpu_capacity(self):
```

3) ממשו את השיטה occupy_available_cpu_capacity אשר מקבלת כקלט את הארגומנט requested_capacity מטיפוס int המייצג כמות של משאבים שיש לשריין. השיטה מניחה כי קיימים משאבים זמינים לשריון לפחות כערך הארגומנט requested_capacity, ומשריינת כמות משאבים זו בצרור אחד של משאבים (או יותר אם יש צורך) ב-cpu_nodes. שריון המשאבים מתבצע ע"י עדכון כמות משאבים שווה ל-requested_capacity כלא זמינים מהצרור הראשון בסדר אשר קיימים בו משאבים זמינים.

בהינתן אותו מופע בסעיף ב-1, המופע יראה כך לאחר הפעלת השיטה עם הקלט requested_capacity השווה ל-7:



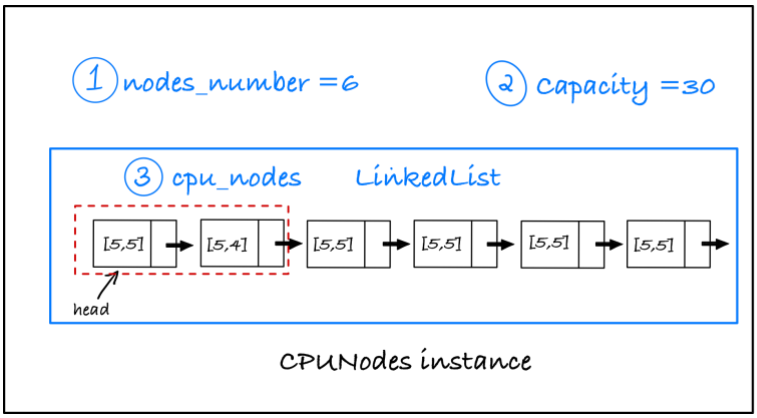

```
def occupy_available_cpu_capacity(self, requested_capacity):
```

4) ממשו את השיטה get_available_cpu_capacity אשר מחזירה את כמות המשאבים הזמינים לשריון בכל צרורות המשאבים של המערכת (ערך מספרי int). לדוגמה, הפעלת השיטה על אותו מופע מהסעיף הקודם (ב-3) תחזיר 23.

```
def get_available_cpu_capacity(self):
```

5) ממשו את השיטה free_occupied_cpu_capacity אשר מקבלת כקלט את הארגומנט freed_capacity המייצג כמות של משאבים שיש לשחרר ולהפוך לזמינה. השיטה מניחה כי קיימים משאבים משוריינים ככמות הארגומנט freed_capacity לכל הפחות באחד מצרורות משאבי המערכת (או שהמשאבים המשוריינים מפוזרים על יותר מצרור יחיד). השיטה הופכת משאבים משוריינים לזמינים בחזרה ע"י עדכון כמות המשאבים הזמינה בצרור המשאבים הראשון שנמצא כמשוריין (או יותר מצרור אחד אם יש צורך). שימו לב כי אין לחרוג מכמות המשאבים המרבית שהצרור "מחזיק".

לדוגמה, הפעלה השיטה על המופע מהסעיף הקודם (ב-4) עם freed_capacity השווה ל-6, תראה כך:




```
def free_occupied_cpu_capacity(self, freed_capacity):
```

6) ממשו את השיטה get_cpu_nodes השיטה מחזירה מצביע לחוליה בראש הרשימה המקושרת בשדה cpu_nodes.

```
def get_cpu_nodes(self):
```

7) ממשו את השיטה __repr__ אשר עושה שימוש ב- __repr__ של רשימה המקושרת כפי שקיבלתם בממשק LinkedList ומחזירה את הייצוג של השדה הפרטי cpu_nodes.

דוגמאות הרצה למחלקה זו יצורפו בקובץ המיועד לכך – example_CPUNodes.txt

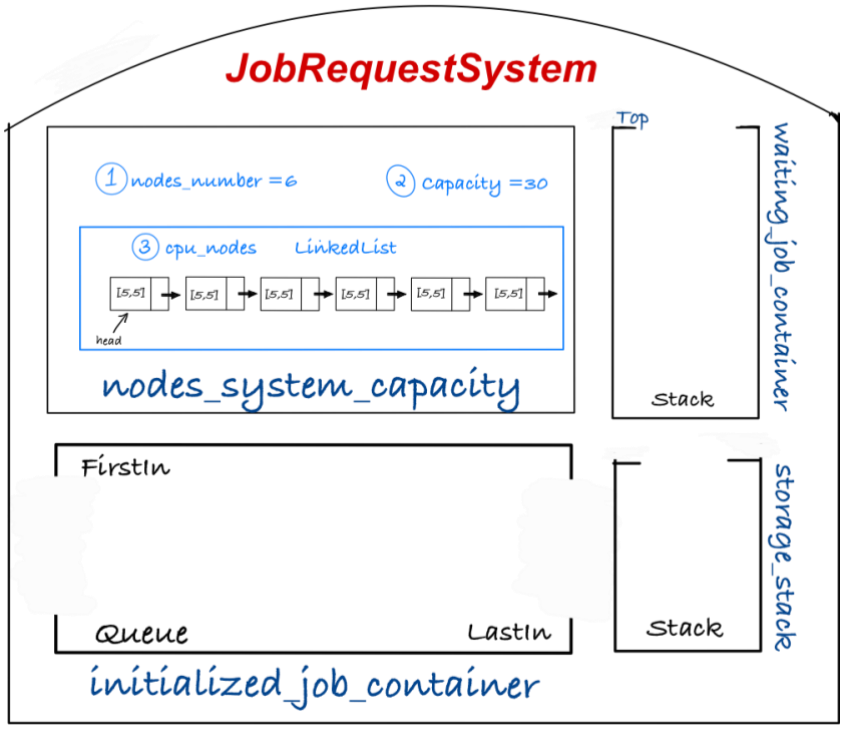
סעיף ג':

המחלקה ***JobRequestSystem*** הינה **המערכת** אשר מבצעת את ניהול המשאבים, שריונם לבקשות המתאימות, ביצוע הבקשות ושחרור המשאבים. המערכת ***JobRequestSystem*** עושה שימוש במופע אחד מהמחלקה CPUNodes ומשתמשת בו כ-"מעטפת" למשאבים העומדים לרשותה. בנוסף, דרך מערכת זו נוצרות הבקשות לשריון המשאבים, מתבצע השריון של המשאבים בהתאם לזמינותם וגם כן "ביצוע" הבקשות ושחרור המשאבים (יוסבר המשך).

ממשו את המחלקה ***JobRequestSystem*** המבצעת את ניהול המשאבים, שריונם לבקשות המתאימות, ביצוע הבקשות ושחרור המשאבים. למחלקה זו **4 שדות פרטיים:**

- השדה nodes_system_capacity מטיפוס CPUNodes המייצג את משאבי המערכת.
- השדה initialized_job_container מטיפוס תור Queue מייצג את אוסף הבקשות ששוריינו.
- השדה waiting_job_container מטיפוס מחסנית Stack מייצג את אוסף הבקשות הנמצאות "במצב המתנה" עד לזמינות משאבים. בקשות במצב המתנה הן ממוינות בכל רגע נתון לפי דירוג ורק אם הדירוג זהה אז לפי סדר הכנסה.
- השדה storage_stack מטיפוס מחסנית (Stack) מייצג משתנה אחסון שיש ברשות המערכת. מחסנית זו מיועדת לשימוש לצרכי מיון הבקשות לפי דירוגן וסדר הכנסתן אשר עתידן להיות במחסנית "מצב המתנה".

למשל מופע מהמחלקה ***JobRequestSystem*** אשר אותחל עם *capacity* כ-30 ו-*nodes_number* ב-*6* יראה כך:



ממשו את השיטות הבאות של המחלקה:

1) ממשו את הבנאי של מחלקה אשר מקבל 2 ארגומנטים capacity ו- nodes_number. הבנאי מאתחל את השדה nodes_system_capacity שהוא מופע מטיפוס CPUNodes וישמש כמשאבי המערכת. בהעדר 2 הארגומנטים

אשר הבנאי מקבל, יש לאתחל את השדה nodes_system_capacity עם nodes_number כ-6 ו-capacity כ-30.
הבנאי של המחלקה מאתחל את שאר השדות של המחלקה כמבני הנתונים הנדרשים ריקים.

```
def __init__(self, nodes_number=5, system_capacity=30):
```

2) ממשו את השיטה request_new_job המקבלת את 3 הארגומנטים job_description, job_rank ו-job_request_resources. השיטה מייצרת מופע של Job בהתאם לארגומנטים ומחזירה אותו. בהעדר שלושת (או כל אחד מהארגומנטים) יש לייצר את המופע של Job עם job_rank כאשר ערך ברירת המחדל שווה ל-1, עם job_description כמחרוזת ריקה ו-job_request_resources כאשר ערך ברירת המחדל שלו שווה ל-1.
בדקו כי מספר המשאבים של הבקשה אינו חורג ממספר המשאבים הכולל שיש למערכת בכלל (בין אם הם זמינים או לא). כלומר, אם ערכו של הארגומנט job_request_resources גדול מסך כל כמות משאבי המערכת (השדה capacity השייך ל- nodes_system_capacity) עליכם לזרוק את החריגה RuntimeError.

```
def request_new_job(self, job_description = "", job_rank = 1,
job_request_resources = 1):
```

3) ממשו את השיטה request_job_resources המקבלת את הארגומנט job_request מטיפוס Job. השיטה בודקת האם קיימים משאבים זמינים לשריון עבור הבקשה job_request במערכת. אם קיימת כמות משאבים זמינה המתאימה לבקשה, על השיטה לשריין את המשאבים האלה עבור הבקשה. שריון המשאבים מתבצע ע"י הפיכתם לאי זמינים לשריון לבקשות אחרות. בקשה משוריינת היא בקשה שהמערכת מחזיקה בתור initialized_job_container.

אם אין מספיק משאבים לבקשה job_request, על השיטה להכניס את הבקשה job_request "במצב המתנה".
שמירת הבקשה job_request במצב המתנה מתבצע ע"י הכנסתה למחסנית waiting_job_container והדפסת ההודעה הבאה למשתמש:

f"Requested resources for Job {job_request.get_id()} must wait for available resources.\n"
f"Consider terminating the current request and trying lower resources\n"
f"Meanwhile you will be waiting for available resources"

כאשר אדום מסמן את התווים הקבועים ושחור את מספר הבקשה וכמות המשאבים הזמינים כעת.

דגשים חשובים:

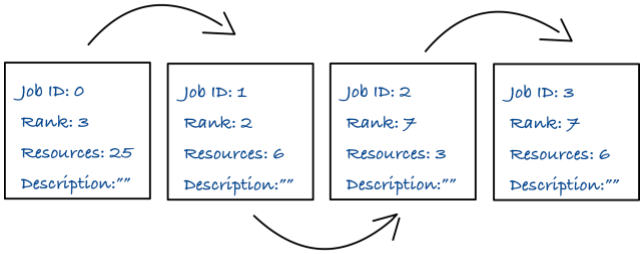
- אם יש מספיק משאבים זמינים לשריון עבור הבקשה job_request אך קיימות במערכת בקשות נוספות שקדמו לה והן נמצאות במצב המתנה (מאחר והן דורשות יותר משאבים מהכמות הזמינה כעת), עליכם להכניס את הבקשה job_request למצב המתנה גם כן. בקשות במצב המתנה במחסנית waiting_job_container ממוינות בכל רגע נתון בהתאם לדירוג של הבקשות ובהתאם לסדר הכנסתן למערכת. עליכם לתת עדיפות לבקשות בעלות הדירוג הגבוה לעומת הנמוך ואלו שהוכנסו קודם לעומת אלו שהוכנסו בשלב יותר מאוחר למערכת.
למשל, אם קיימות במערכת 2 בקשות במצב המתנה ולהן יש את הדירוגים 1 ו- 5 (הבקשה בעלת הדירוג 5 תמיד תהיה בראש המחסנית waiting_job_container- כאשר היא והבקשה בעלת הדירוג 1 בלבד במצב

המתנה), וכעת יש להכניס בקשה נוספת בעלת דירוג 3 למצב המתנה, יש להכניס אותה למחסנית ההמתנה בהתאם לדירוגה ולסדר ההכנסה שלה. כעת המחסנית של בקשות במצב המתנה waiting_job_container תכיל את הבקשות בסדר הבא (1->3->5->ראש המחסנית).

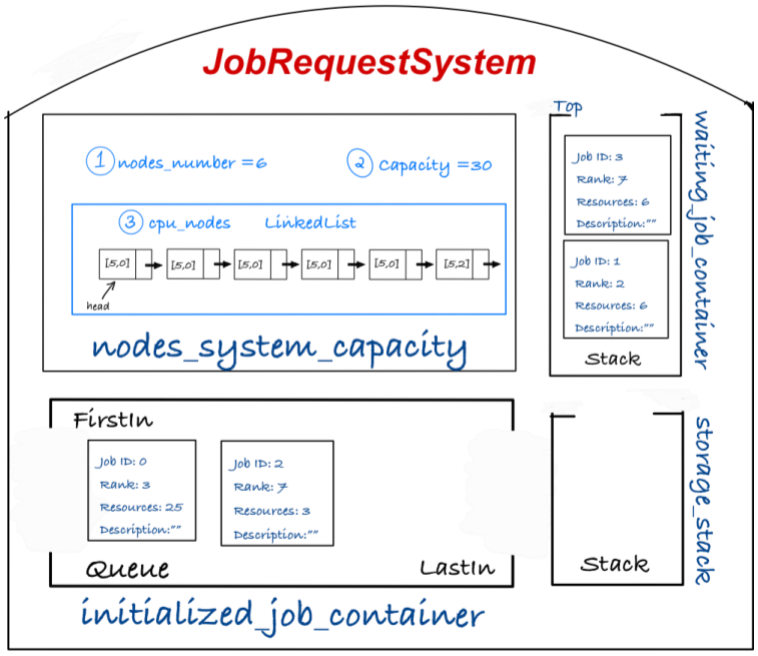
- אם הבקשה job_request היא בעלת העדיפות הגבוהה ביותר בין כל הבקשות במצב המתנה ולמערכת יש מספיק משאבים זמינים לשריון עבורה, יש לשריין אותה.

```
def request_job_resources(self, job_request):
```

להלן איור סכמתי לתהליך שריון של ארבעת מופעי Job הבאים שיש לשריין עבורם משאבים בסדר הנתון הבא:



תמונת מצב למערכת תראה ככה:



הבקשה 0 שוריינה בהצלחה מאחר והיו מספיק משאבים זמינים עבורה והיא הוכנסה לתור הבקשות המשוריינות initialized_job_container. לעומת זאת, הבקשה 1 נכנסה ל-"מצב המתנה" (מחסנית waiting_job_container) כי מספר המשאבים הזמינים לה (5) יותר נמוך ממספר המשאבים אשר היא דורשת על מנת שתשוריין (6). הבקשה 2 אינה במצב המתנה והיא הוכנסה לתור הבקשות המשוריינות מאחר והדירוג שלה גבוה מבקשה 2 ונדרשים עבור שריונה רק 3 משאבים. בקשה 3 נכנסה למצב המתנה אבל היא בראש המחסנית waiting_job_container כי דירוגה יותר גבוה מבקשה 1 אך אין מספיק משאבים לשריון עבורה.

4) ממשו את השיטה execute_job אשר "מבצעת" את הבקשה הראשונה בראש התור initialized_job_container המכיל את הבקשות אשר שוריינו עבורם משאבים במערכת. ביצוע הבקשה הראשונה בתור מתרחש ע"י שליפתה מהתור. בנוסף לכך, השיטה תדפיס את ההודעה הבאה:

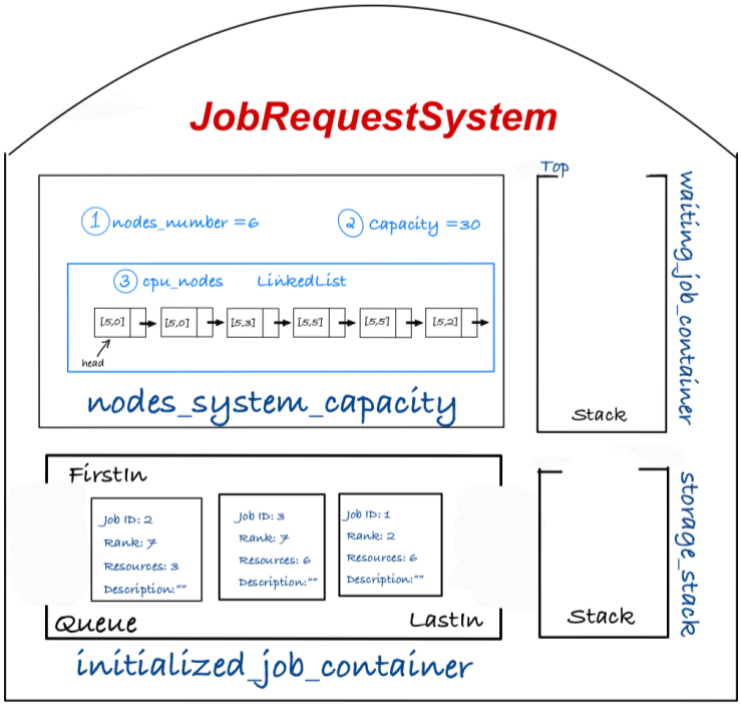
"The current job with the id: {job_id} is being executed....\n"
"The Job with the id {job_id } has finished executing successfully."

כאשר אדום מסמן את התווים הקבועים ושחור את המספר המזהה של הבקשה "שמבוצעת" כעת.

ביצוע הבקשה הראשונה בראש התור initialized_job_container משחרר כמות משאבים זהה לכמות ששוריינה עבורה והופך אותם לזמינים בחזרה עבור בקשות אחרות. לאחר שחרור המשאבים, השיטה בודקת האם המשאבים ששוחררו כעת במערכת מאפשרים שריון של בקשות הנמצאות במצב המתנה ובהתאם לזמינות המשאבים משריינת אותם.

```
def execute_job(self):
```

למשל, בהמשך לאיור הקודם בעמוד 9, לאחר "ביצוע" הבקשה הראשונה בתור initialized_job_container, המערכת תראה ככה:



ביצוע הבקשה 0 שחרר 25 משאבים, וכעת יש מספיק משאבים לשתי הבקשות במצב המתנה. לכן, הועברו שתיהן לתור הבקשות המשוריינות ושוריינו משאבים בהתאם להן במערכת.

5) ממשו את השיטה terminate_job המקבלת את הארגומנט job_request_id מטיפוס int. השיטה מחפשת את הבקשה בעלת המספר הייחודי job_request_id אשר נמצאת במצב המתנה במחסנית waiting_job_container ומוציאה אותו מהמחסנית. השיטה מחזירה את הבקשה בעלת מספר ייחודי זה (מטיפוס Job).
שימו לב כי המחסנית של בקשות במצב המתנה waiting_job_container חייבת להיות ממוינת בכל רגע נתון לפי דירוג הבקשות ואם הדירוג זהה אז לפי סדר ההכנסה למערכת. על מנת למיין את מחסנית מצב ההמתנה עומדת לרשותכם מחסנית האחסון בשדה storage_stack כמחסנית עזר.
תניחו כי הבקשה בעלת המספר הייחודי job_request_id קיימת בהכרח במערכת "במצב המתנה".

```
def terminate_job(self, job_request: Job):
```

6) ממשו את השיטה total_waiting_requested_job_resources המחשבת ומחזירה את סכום המשאבים הכולל שדורשות כל הבקשות במצב המתנה (מטיפוס int). המחסנית של בקשות במצב המתנה waiting_job_container חייבת להיות ממוינת בכל רגע נתון לפי דירוג הבקשות וסדר הכנסתן למערכת.

```
def total_waiting_requested_job_resources(self):
```

דוגמאות הרצה למחלקה זו יצורפו בקובץ המיועד לכך – example_JobRequestSystem.txt

שאלה 2:

בשאלה זו עומד לרשותכם המימוש של עץ בינארי (קובץ BinaryTree.py) אשר תרחיבו אותו ע"י מימוש של מספר שיטות נוספות. בשאלה זו, המפתח והערך של כל קודקוד בעץ זהים.

דוגמאות הרצה עם הפלטים הרלוונטיים לכל סעיפי השאלה מצורפות בקובץ example_BinaryTree.txt.

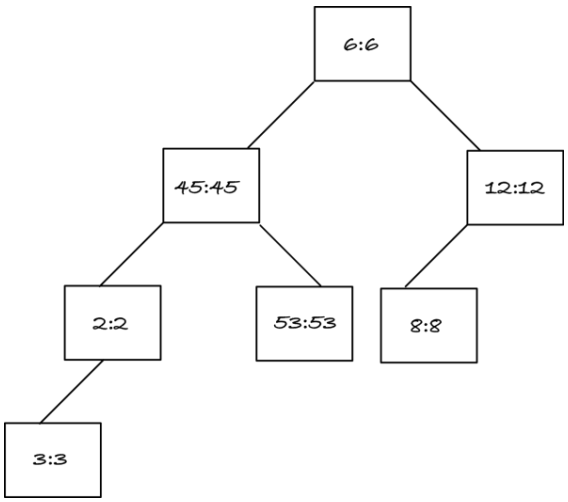
סעיף א:

ממשו את השיטה root_to_leaf_path_rec המחזירה רשימה מקוננת המייצגת את המסלולים משורש העץ אל כל אחד מהעלים. יש לממש את פונקציה זו בעזרת רקורסיה בלבד. פתרון איטרטיבי בסעיף זה יפסל.

```
def root_to_leaf_path_rec(self):
```

דוגמה 1:

בהינתן העץ הבינארי הבא:

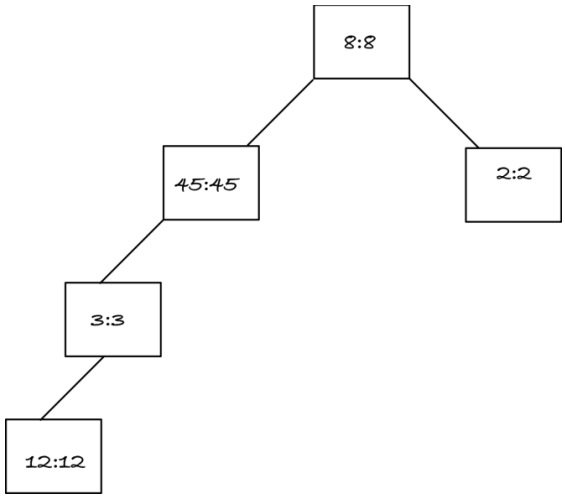


תוחזר הרשימה הבאה:

```
[[6,45,2,3],[6,45,53],[6,12,8]]
```

דוגמה 2:

בהינתן העץ הבינארי הבא:



תוחזר הרשימה הבאה:

```
[[8,45,3,12],[8,2]]
```

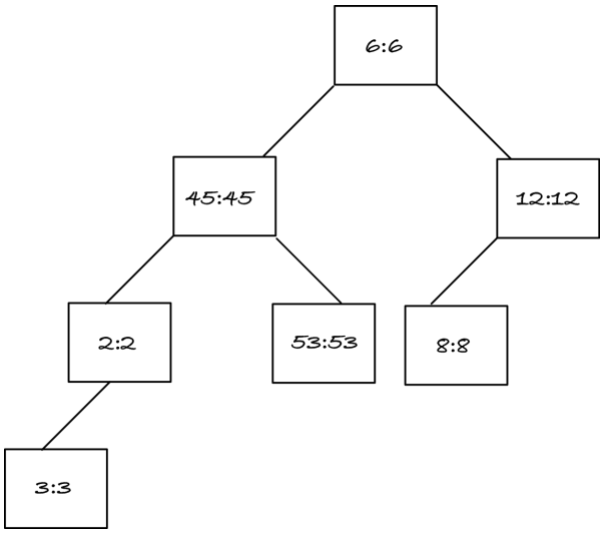
סעיף ב:

ממשו את השיטה root_to_leaf_path_iter המחזירה רשימה מקוננת המייצגת את המסלולים משורש העץ אל כל אחד מהעלים. עליכם לממש את הפונקציה הזו באופן איטרטיבי בעזרת שימוש במחסנית עזר.

```
def root_to_leaf_path_iter(self):
```

דוגמה 1 :

בהינתן העץ הבינארי הבא :

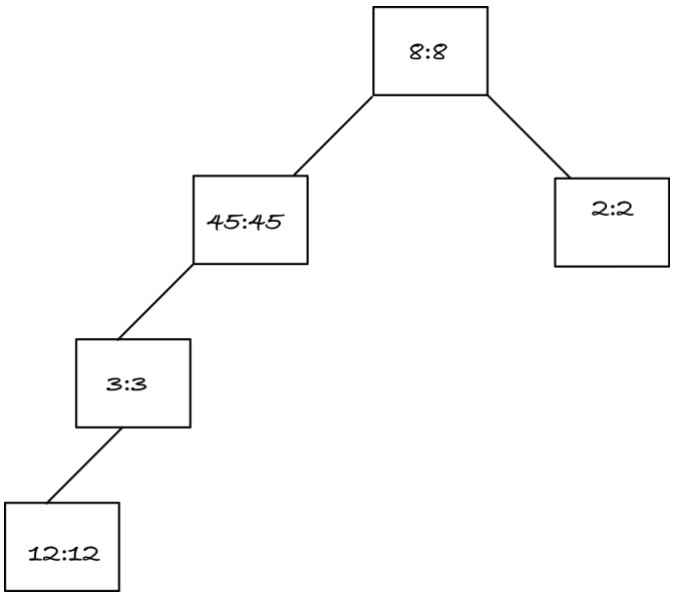


תוחזר הרשימה הבאה :

```
[[6,45,2,3],[6,45,53],[6,12,8]]
```

דוגמה 2 :

בהינתן העץ הבינארי הבא :



תוחזר הרשימה הבאה :

```
[[8,45,3,12],[8,2]]
```

סעיף ג:

נגדיר שקודקוד x הוא שכן של קדקוד y, אם שני הקודקודים נמצאים באותו מרחק מהשורש והם אינם צאצאים של אותו אב ישיר.

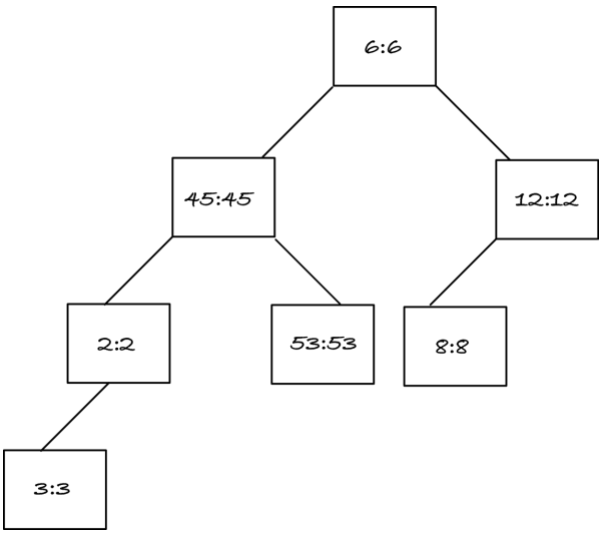
ממשו את השיטה print_all_neighbors אשר מקבלת ארגומנט אחד node המהווה קודקוד במורד העץ (מטיפוס TreeNode). השיטה מדפיסה את כל שכני הקודקוד node במורד העץ המושרש ב-root.

אין צורך לבצע בדיקת תקינות קלט, תניחו כי הקודקוד node קיים במורד העץ הבינארי.

```
def print_all_neighbors(self, node):
```

דוגמה 1:

בהינתן הקודקוד 8 והעץ הבינארי הבא:



יודפסו ערכיהם של שני הקודקודים 53 ו-2 מאחר ושלושת הקודקודים 8, 53, ו-2 הם באותו מרחק מהשורש (באותה רמה) ורק שני הקודקודים 53 ו-2 אינם צאצאים של אותו אב ישיר של הקודקוד 8.

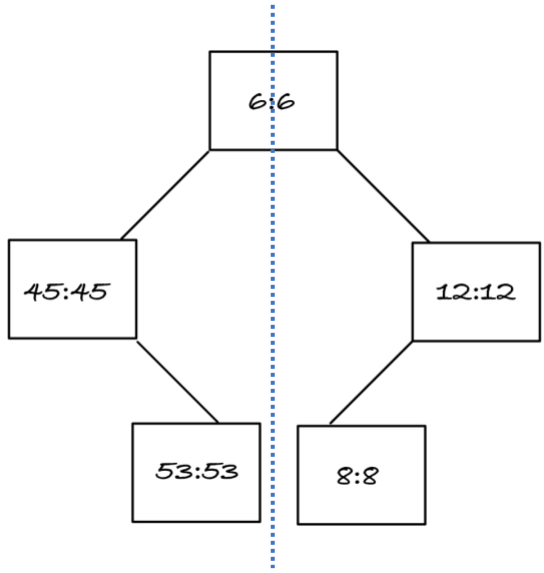
דוגמה 2:

בהינתן אותו עץ והקודקוד 2 יודפס הקודקוד 8 בלבד מאחר ורק הוא באותו מרחק מהשורש כמו קודקוד 2. שימו לב כי שני הקודקודים 53 ו-2 הם צאצאים של אותו אב ישיר ולכן לא יודפס 53.

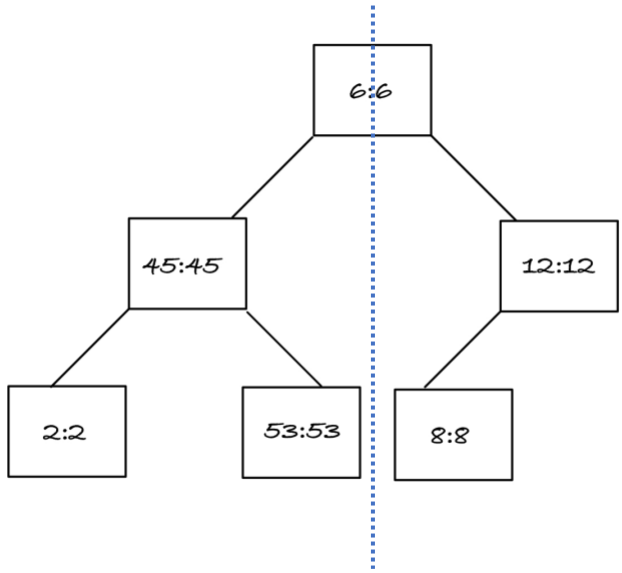
סעיף ד:

נגדיר עץ בינארי כעץ סימטרי אם המבנה של תת העץ הימני זהה למבנה של תת העץ השמאלי.

למשל העץ בדוגמה למטה הוא סימטרי, מאחר ולשניהם מבנה זהה.



לעומת זאת, העץ בדוגמה הבאה אינו סימטרי. לתת עץ השמאלי יש קודקוד שאין לו השתקפות בתת עץ הימני (הקודקוד עם הערך 2:2).



ממשו את השיטה is_symmetric_tree אשר בודקת האם מבנהו של העץ הוא סימטרי. השיטה מחזירה True אם העץ סימטרי ו-False אחרת.

```
def is_symmetric_tree(self):
```